

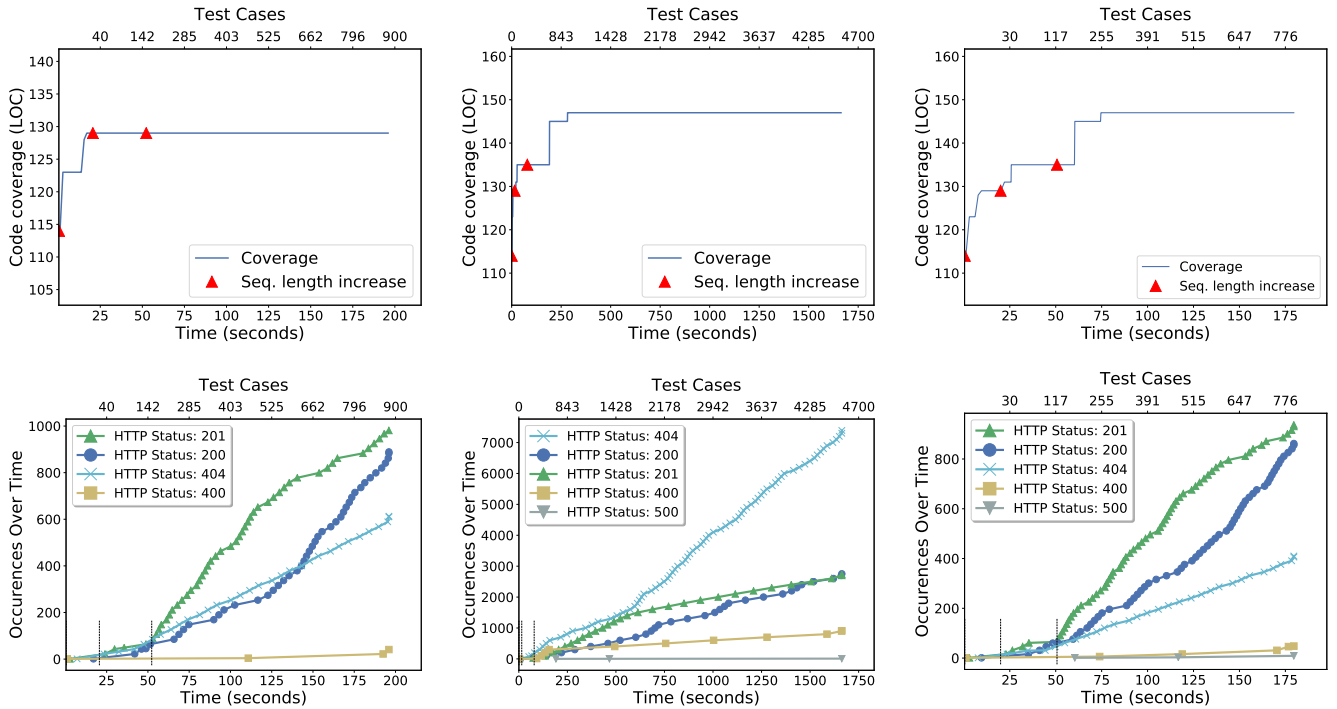


Fig. 4: **Blog Posts Service Code Coverage and HTTP Status Codes Over Time.** Shows the increase in code coverage over time (top) and the cumulative number of HTTP status codes received over time (bottom), for the simple blog posts service. *Left:* *RESTler* ignores dependencies among request types. *Center:* *RESTler* ignores dynamic feedback. *Right:* *RESTler* utilizes both dependencies among request types and dynamic feedback. When leveraging both techniques, *RESTler* achieves the best code coverage and finds the planted 500 “Internal Server Error” bug with the least number of tests.

API	Total Requests	Seq. Len.	Coverage Increase	Tests	seqSet Size	Dynamic Objects
Commits	11	1	598	1	1	1
		2	1108	7	5	10
		3	1196	250	46	521
		4	1760	2220	1341	6577
		5	1760	3667	20679	12518
Branches	7	1	598	1	1	1
		2	1089	8	6	11
		3	1172	58	44	107
		4	1182	576	387	1279
		5	1185	3644	5528	9336
Issues	22	1	816	37	37	37
		2	1163	2444	1839	4245
		3	1163	4156	15658	8870
Repos	10	1	598	1	1	1
		2	1117	97	65	206
		3	1181	5153	2194	15472
Groups	50	1	887	39	39	38
		2	1177	3508	3360	5204
		3	1177	4817	79518	8946
Projects	48	1	934	42	41	38
		2	1192	1870	1781	3343
		3	1203	3226	18173	7374

TABLE I: **Testing Common GitLab APIs with *RESTler*.** Shows the increase in sequence length, code coverage, tests executed, seqSet size, and the number of dynamic objects being created using BFS, until a 5-hours timeout is reached. Longer request sequences gradually increase service-side code coverage.

with the test-generation algorithm of Figure 3 using BFS. For each experiment we run *RESTler* with a 5-hours timeout and limit the number of fuzzable primitive-type combinations to

maximum 1,000 combinations per request. Between experiments, we reboot the entire GitLab service to restart from the same initial state. For each API group, as time goes by, Table I shows the increase (going down) in the sequence length, code coverage, tests executed, seqSet size, and the number of dynamic objects created, until the 5-hours timeout is reached. **Code Coverage.** We collect code coverage data by configuring Ruby’s `Class: TracePoint` hook to trace GitLab’s `service/lib` folder. Table I shows the cumulative code coverage achieved after executing all the request sequences generated by *RESTler* for each sequence length, or until the 5-hours timeout expires. The results are incremental on top of 16,836 lines of code executed during service boot.

From Table I, we can see that longer sequence lengths consistently lead to increased service-side code coverage. This is the desired behaviour, especially for small sequence lengths, as some of the service functionality can only be exercised after at least a few requests are executed. As an example, consider the GitLab functionality of “selecting a commit”. According to GitLab’s specification, selecting a commit requires two dynamic objects, a *project-id* and a *commit-id*, and the following dependency of requests is implicit: (1) a user needs to create a project, (2) use the respective *project-id* to post a new commit, and then (3) select the commit using its *commit-id* and the respective *project-id*. Clearly, this operation can only be performed by sequences of three requests or more. For the Commit APIs, note the gradual increase in coverage from 598 to 1,108 to 1,196 lines of code for sequence lengths of one,

API	Total Requests	Time (hrs)	BFS			BFS-Fast			RandomWalk		
			Len.	Coverage	seqSet	Len.	Coverage	seqSet	Len. (restarts)	Coverage	seqSet
Commits	11 (*11)	1	4	1202		7	1697		13 (16)	1285	
		3	5	1760		9	1731		13 (35)	1295	
		5	5	1760	20679	12	1731	33	13 (56)	1303	1
Branches	7 (*2)	1	5	1182		21	1154		15 (24)	1182	
		3	5	1185		37	1178		19 (92)	1187	
		5	5	1185	5528	47	1178	11	22 (158)	1208	1
Issues	22 (*82)	1	2	1150		2	1086		10 (1)	770	
		3	3	1163		4	1551		10 (1)	770	
		5	3	1163	15658	5	1570	26	16 (2)	847	1
Repos	10 (*24)	1	3	1127		5	1141		10 (29)	1195	
		3	3	1127		7	1141		13 (88)	1231	
		5	3	1181	2194	8	1161	64	13 (142)	1231	1
Groups	50 (*2)	1	2	961		6	1275		19 (41)	1167	
		3	3	1177		11	1275		19 (120)	1250	
		5	3	1177	79518	14	1275	130	22 (186)	1283	1
Projects	48 (*4)	1	2	1006		5	1318		4 (3)	889	
		3	2	1053		11	1319		22 (31)	1024	
		5	3	1203	18173	15	1319	171	22 (45)	1273	1

TABLE II: **Comparison of BFS, BFS-Fast and RandomWalk over Time.** Shows the maximum sequence length, the increase in lines of code covered (excluding service-boot coverage), and the seqSet size with each search strategy after 1, 3, and 5 hours. The second column shows the total number of requests in each API along with the average feasible request renderings (*). Although *BFS* covers slightly more lines of code, *BFS-Fast* and *RandomWalk* reach deeper request sequences and maintain a much smaller seqSet size.

two, and three, respectively. Most notably, for the Branches API, service-side code coverage keeps gradually increasing for sequences of length up to five, and reaches 1,185 lines when the 5-hours limit expires.

Tests, Sequence Sets, and Dynamic Objects. In addition to code coverage, Table I also shows the increase in the number of tests executed, the size of seqSet after the RENDER function returns (line 10 of Figure 3), and the number of dynamic objects created by *RESTler*. All those numbers are quickly growing since the search space also grows quickly due to the exhaustive nature of the BFS search strategy.

Nevertheless, we emphasize that without the two key techniques evaluated in Section V-B this growth would be much worse. For instance, for the Commit API, the SeqSet size is 20,679 and there are 12,518 dynamic objects created by *RESTler* for sequences of length up to five. By comparison, since the Commits API has 11 request types with an average of 4 rendering combinations, the number of all possible rendered request sequences of up to length four is already more than 164 millions, and a naive brute-force enumeration of those would already be untractable. Still, even with the two core techniques used in *RESTler*, the search space explodes quickly, and we evaluate other search strategies next.

D. Search Strategies

We now present results of experiments comparing the BFS, BFS-Fast, and RandomWalk search strategies defined in Section III (Q3). For each search strategy, Table II shows the maximum sequence length, the increase in lines of code covered (excluding service-boot coverage) after 1, 3, and 5 hours, and the size of the seqSet when the 5-hours timeout is

reached. For the RandomWalk search strategy the total number of restarts is also shown in parenthesis.

First, we compare BFS with BFS-fast. We observe that after five hours, BFS achieves better coverage than BFS-Fast in Commits, Branches, and Repos. These groups of APIs have relatively fewer requests and BFS delivers better coverage by exercising all feasible request sequences. However, BFS does not scale well in APIs with relatively more requests, such as Issues, Groups, and Projects. As shown in Table II after 5 hours for Issues, Groups, and Projects, BFS is still exploring sequences of length 3 while BFS-Fast is exploring sequences of length 5, 14, and 15, respectively. BFS-Fast scales better in APIs with many request because, unlike BFS, it does not explore all feasible request sequences but instead appends each request to at most one sequence in each generation. BFS-Fast maintains a smaller seqSet, and explores deeper sequences and grows coverage faster in Issues, Groups, and Projects.

We now compare BFS with RandomWalk. By construction, RandomWalk does not guarantee full grammar coverage since it appends each request to one random sequence in each generation. As shown in Table II, RandomWalk maintains a small seqSet at any time, by construction. Furthermore, after 5 hours RandomWalk explores considerably deeper request sequences compared to BFS and, in most cases, compared to BFS-Fast. RandomWalk also delivers the best coverage after 5 hours in Branches, Repos, and Groups.

On the other hand, in Issues, we observe that after 5 hours RandomWalk explores sequences of length 16 and the coverage increase is 847 lines. In the same time-frame, BFS explores sequences of length 3 but the coverage increase is 1,163 and BFS-Fast explores sequences length 5 with a

API	BFS	BFS-Fast	Random-Walk	Intersection	Union
Commits	5	1	5	1	5
Branches	7	7	7	5	8
Issues	0	1	1	0	1
Repos	2	3	3	2	3
Groups	0	0	2	0	2
Projects	2	1	3	1	3
Total	16	13	21	9	22

TABLE III: **Bug Buckets found by BFS, BFS-Fast, and RandomWalk after Five Hours.** Shows the sets of bugs found by each search strategy in each API. In total: *RESTler* found 22 new bugs.

coverage increase of 1,570. Compared to all other APIs shown in Table II, Issues have 82 feasible request renderings on average. This is relatively large and in such a case, with many feasible request renderings, the breadth of the search achieved by RandomWalk is small (*e.g.*, after 5 hours there are only 2 restarts). Consequently, the search remains focused on a very restricted subspace which reflects poorly on coverage.

In practice, both controlling the size of `seqSet`, when facing broader search spaces due to large APIs with many requests or when reaching greater depths, and maintaining some breadth when extending request sequences seem key to delivering better code coverage. Nevertheless, the ultimate goal is to find bugs, and maximizing code coverage is just a heuristic to try to reach that goal.

E. Bug Bucketization

Before discussing real bugs found with *RESTler*, we introduce a bucketization scheme to cluster similar 500 “Internal Server Errors”. When fuzzing, different instances of a same bug are often found repeatedly. Since all the bugs found have to be inspected by the user, it is therefore important in practice to facilitate this analysis by identifying likely-redundant instances of a same unique bug.

In our context, we define a *bug* as a 500 HTTP status code being received after executing a request sequence. Thus, every bug found is associated with the request sequence that was executed to find it. Given this property, we use the following bucketization procedure for the bugs found by *RESTler*:

Whenever a new bug is found, we compute all non-empty suffixes of its non-rendered request sequence² (starting with the smallest one) and check whether some suffix is a previously-recorded sequence leading to a bug found earlier. If there is a match, the new bug is added to the bucket of that previous bug. Otherwise, a new bucket is created with the new bug and its request sequence.

When using BFS or BFS-Fast, this bucketization scheme will identify bugs by the shortest sequence necessary to find it.

Table III shows the sets of bug buckets found by each search strategy, after five hours, in each GitLab API group. To demonstrate the overlap between the bugs reported by each

method, the last two columns show the intersection and the union of the bug buckets. In the context of these experiments, *RESTler* found 22 new *unique* bugs, after running each search strategy for 5 hours on each API group.

RandomWalk stands out in Table III by finding the most bugs: 21 compared to 16 and 13 for BFS and BFS-Fast respectively. It is particularly intriguing that RandomWalk finds as many bugs as BFS and BFS-Fast combined in Commits and in Issues APIs because in these APIs RandomWalk delivers relatively little coverage. After 5 hours, in commits, RandomWalk finds as many bugs as BFS and more than BFS-Fast. At the same time, RandomWalk delivers less code coverage than each of BFS and BFS-Fast in Commits (see Table II). Similarly, RandomWalk finds 1 bug in Issues, while BFS finds none and BFS-Fast also finds one. Yet, again, RandomWalk achieves less code coverage than each of BFS and BFS-Fast in Issues. The differences between BFS and BFS-Fast are less striking. BFS finds more bugs in Commits, while BFS-Fast finds more bugs in Issues and Repos.

Overall, within the 5-hours time-frame of our experiments, RandomWalk finds more bugs than BFS or BFS-Fast despite the fact that it does not always deliver the best coverage. It is unclear how this generalizes to longer fuzzing sessions or to other APIs. Yet, it becomes apparent that coverage increase should not always dictate the selection of a search strategy because different search strategies may be complementary within a large search space. Next, we discuss details of the bugs found with *RESTler* in GitLab and the total number of bugs found when running longer fuzzing experiments.

VI. NEW BUGS FOUND IN GITLAB

During all our fuzzing experiments with *RESTler* on our local GitLab deployment, we found a total of 28 new unique bugs. All bugs were easily reproducible, disclosed to GitLab developers, confirmed and fixed. Due to space limitation, we describe only 2 of these bugs, to give the reader a flavor of what those bugs look like and how they were found. (See [16], [17], [18], [19] for other examples of bugs found.)

Example 1: Bug in Commits API. One of the bugs found by *RESTler* in the Commits API is triggered when a user tries to cherry-pick a commit to a branch with an empty name. Due to incomplete input validation, an invalid branch name can be passed between two different layers of abstraction as follows: The ruby code that checks if a target branch exists, invokes a native C function whose return value is expected to be either NULL or an existing entry. However, if an unmatched entry type (*e.g.*, an empty string) is passed to the native function, an exception is raised. This exception is unhandled by the higher-level ruby code, and therefore it causes a 500 “Internal Server Error”. The bug can be reproduced by (1) creating a project, (2) creating a new branch (in addition to master branch which is created by default), (3) posting a valid commit with action “create” in the branch created in (2), and (4) cherry-picking the commit to a branch whose name is set to the empty string.

Example 2: Bug in Branches API. Another bug, found by *RESTler* in the Branches API, is triggered when a user

²A request sequence of length n has n suffixes of length 1, 2, ..., n .

tries to edit a branch of a recently deleted project. The bug is due to invalid serialization of operations which results in an database entry update using an invalid foreign key of a deleted project. Since the project-id (foreign key) is not present in the respective “projects” table, a *PG::ForeignKeyViolation* exception causes a 500 “Internal Server Error”. The bug can be reproduced by (1) creating a project, (2) creating a branch, (3) deleting the project created in (1), and (4) quickly editing the branch of the deleted project.

From the above bug descriptions, we see a two-fold pattern. First, *RESTler* produces a sequence that exercises the target service deep enough so that it reaches a particular valid “state”. Second, while the service is in such a state, *RESTler* produces an additional request with an unexpected fuzzed value (e.g., an empty string) or an unexpected action (e.g., edit a branch of a recently deleted project). Most bugs found by *RESTler* require a combination of these two features in order to be found.

VII. EXPERIENCES WITH PUBLIC CLOUD SERVICES

In this section, we describe our preliminary experiences running *RESTler* on three Azure [29] services and one Microsoft Office365 [30] service. The services we fuzzed primarily perform resource management and real-time data aggregation. Swagger specifications for these services are publicly available and published by Microsoft on GitHub.

While still in an early stage of development, *RESTler* found new bugs in all of these services. These bugs range from mis-handled invalid inputs (e.g., using a wrong ID or enum value), executing operations in invalid states (e.g., updating a resource that no longer exists), and inconsistent parameter validations (e.g., using a valid request body with incorrect metadata). Although we cannot disclose detailed descriptions of these bugs, we emphasize that all bugs found by *RESTler* so far have been confirmed and fixed by Microsoft service developers. Indeed, “500 Internal Server Errors” are server state corruptions that may severely damage service health and security: it is safer to fix these rather than risk a live incident with unknown consequences.

During this effort, we faced a number of challenges unique to public cloud services, including resource quota limitations, short-lived access tokens, and complex API dependencies beyond the canonical REST API structure with application-specific resource values and naming schemes. We describe the extensions made to *RESTler* to address these challenges.

Resource Quotas. Production services that run in public clouds are deployed with default resource quotas. Once quotas are reached, *RESTler*’s core algorithm will continue to try request sequences containing requests that can no longer succeed due to exceeded quotas (since these requests were valid in prior tests and generated lots of new resources), which impedes progress. This challenge is unique to public cloud deployments, contrary to self-contained deployments where one can easily control and reconfigure resource quotas. To address this problem, we implemented a garbage collector (GC) in *RESTler*. The GC runs as a separate thread that monitors the creation of dynamic resources over time and

periodically deletes dynamic objects that are no longer used in order to avoid exceeding resource quotas. This allows *RESTler* to continuously test new sequences for hours or days without hitting resource-quota-related errors.

Short-lived Access Tokens. Unlike in self-contained deployments where an admin can pre-populate static or long-lived authentication tokens, public cloud services use short-lived, refreshable authentication tokens. Usually, a public endpoint, accessible with some type of static credentials (e.g., a username-password pair or a master token) and service-specific logic, generates fresh, short-lived access tokens. The latter are added in the header of HTTPS requests. Since different services may require custom logic to access their public authentication endpoints, *RESTler* provides an authentication hook which periodically executes a user-provided piece of code (e.g., a script) and propagates fresh values in the pool of refreshable authentication tokens.

Application-specific Naming Schemes. As discussed in Section II, *RESTler* performs a light-weight static analysis of a Swagger specification to infer dependencies among requests of the target REST API. However, part of a target API may not be fully REST compliant, or the specification may be incomplete, and consequently the inferred dependencies will also be incomplete. To address this challenge, *RESTler* supports annotations, which can be added directly to the specification (as Swagger extensions), in order to explicitly declare dependencies, as well as resource-specific mutations, which can be used for the creation of resources that require some custom format (e.g., an IP address). These two features have proven useful in practice because Azure services use PUT requests to create resources whose user-provided names are passed as URL parameters and, after successful creation, are also returned in the response. For this scenario, one can use resource-name-specific mutations to indicate that a PUT request should create a resource named in a custom format, and then use that name to identify the corresponding dynamic object in subsequent requests.

VIII. RELATED WORK

HTTP Fuzzers. Since REST API requests and responses are transmitted over the HTTP protocol, HTTP-fuzzers can be used to fuzz REST APIs. Fuzzers like Burp [8], Sulley [38], BooFuzz [7], or the commercial AppSpider [4] and Qualys’s WAS [34], can capture/replay HTTP traffic, parse HTTP requests/responses and their contents (like embedded JSON data), and then fuzz those using either pre-defined heuristics [4], [34] or user-defined rules [38], [7]. Tools to capture, parse, fuzz, and replay HTTP traffic have recently been extended to leverage Swagger specifications in order to parse HTTP requests and guide their fuzzing [4], [34], [41], [3]. Compared to those tools, the main originality of *RESTler* is the lightweight static analysis of Swagger specifications in order to infer dependencies among request types, which in turn allows *RESTler* to automatically generate sequences of requests that exercise the business logic exposed by the API in a *stateful manner* and *without pre-recorded HTTP traffic*.

Feedback-directed Test Generation. The dynamic feedback *RESTler* uses to prune invalid requests from the search space (line 32 in Figure 3) is similar to the feedback used in Randoop [32]. However, the Randoop search strategy (in particular, search pruning and ordering) is different from the three search strategies considered in our work, and the Randoop optimizations related to object equality and filtering are not relevant in our context. *RESTler*’s dependency analysis is also related to the analysis of type dependencies performed by the Randoop algorithm [32] for typed object-oriented programs. However, unlike in the Randoop work, dynamic objects in Swagger specifications are implicitly declared and untyped (*e.g.*, authentication tokens or service-specific resources). When a Swagger specification is not complete or *RESTler* cannot infer object types correctly, *RESTler* supports annotations (see Section VII) that the user can use to fix and control *RESTler*’s behavior. In the future, it would be interesting to allow richer user annotations in order to easily specify complex service-specific types as well as their properties, in the spirit of code contracts [28], [5].

Model-based Testing. Our BFS-Fast search strategy is inspired by test generation algorithms used in model-based testing [42], whose goal is to generate a minimum number of tests covering, say, every state and transition of a finite-state machine model (*e.g.*, see [43]) in order to generate a test suite to check conformance of a (blackbox) implementation with respect to that model. BFS-Fast is also related to algorithms for generating tests from an input grammar while covering all its production rules [26]. Indeed, in our context, BFS-Fast provides, by construction, a full grammar coverage up to the given current sequence length. The number of request sequences it generates is not necessarily minimal, but that number was always small, hence manageable, in our experiments so far.

Grammar-based Fuzzing. General-purpose grammar-based fuzzers like Peach [33] and SPIKE [37], among others [39], are not Swagger-specific but can also be used to fuzz REST APIs. With these tools, however, the user has to *manually* construct an API-specific input grammar, often encoded directly by code specifying what and how to fuzz, similar to the code shown on the right of Figure 2. By contrast, *RESTler* *automatically* generates an input grammar from a Swagger specification, and its fuzzing rules are determined separately and automatically by the algorithm of Figure 3.

Automatically learning input grammars from input samples is another complementary research area [25], [6], [24], [36]. *RESTler* currently relies on a Swagger specification to represent a service’s input space and it learns automatically how to prune invalid request sequences by analyzing service responses. Still, a Swagger specification could be further refined given representative unit tests or live traffic in order to focus the search towards specific areas of the input space. For REST services without a Swagger specification, it would be worth investigating how to automatically infer it by using machine learning on runtime traffic logs or static analysis on the code implementing the API.

Whitebox Fuzzing. Grammar-based fuzzing can also be com-

bined [27], [21] with whitebox fuzzing [23], which uses dynamic symbolic execution [22], [9], constraint generation and solving in order to generate new tests exercising new code paths. In contrast, *RESTler* is currently purely blackbox: the inner workings of the service under test are invisible to *RESTler* which only sees REST API requests and responses. Since cloud services are usually complex distributed systems whose components are written in different languages, general symbolic-execution-based approaches seem problematic, but it would be worth exploring this option further. For instance, in the short term, *RESTler* could be extended to take into account alerts (*e.g.*, assertion violations) reported in back-end logs in order to increase chances of finding interesting bugs and correlating them to specific request sequences.

Penetration Testing. In practice, the main technique used today to ensure the security of cloud services is the so-called “penetration testing”, or *pen testing*, which means security experts review the architecture, design and code of cloud services from a security perspective. Since pen testing is labor intensive, it is expensive and limited in scope and depth. Fuzzing tools like *RESTler* can partly automate the discovery of specific classes of security vulnerabilities, and are complementary to pen testing.

IX. CONCLUSION

RESTler is the first automatic tool for stateful fuzzing of cloud services through their REST APIs. While still in early stages of development, *RESTler* was able to find 28 bugs in GitLab and several bugs in each of the four Azure and Office365 cloud services tested so far. Although still preliminary, our results are encouraging. How general are these results? To find out, we need to fuzz more services through their REST APIs and check more properties to detect different kinds of bugs and security vulnerabilities. Indeed, unlike buffer overflows in binary-format parsers, use-after-free bugs in web browsers, or cross-site-scripting attacks in web-pages, it is still unclear what security vulnerabilities might hide behind REST APIs. While past human-intensive pen testing efforts targeting cloud services provide evidence that such vulnerabilities do exist, this evidence is still too anecdotal. New automated tools, like *RESTler*, are needed for more systematic answers. How many bugs can be found by fuzzing REST APIs? How security-critical will they be? This paper provides a clear path forward to answer these questions.

X. ACKNOWLEDGEMENTS

We thank William Blum, Dave Tamasi and David Molnar for their helpful comments, and the whole Microsoft Security Risk Detection team for their support. We also thank Albert Greenberg, Mark Russinovich and John Walton, from Microsoft Azure, for encouraging us to pursue this line of work. Finally, we thank the GitLab and Microsoft developers we interacted with, for graciously acknowledging, discussing and fixing the bugs found during this work.